

THE COMMUNICATION SYSTEMS

1. SPI PROTOCOL - serial peripheral interface

Serial means synchronous, peripheral are the i/o devices

The i/o devices can be sensors, displays etc

In our project we are using spi protocol for the fpga and main communication

Lets explain what spi is in detail

SPI Bus Architecture

SPI follows a **master-slave architecture**, where a single **master** device controls one or more **slave** devices. The communication occurs through four main lines:

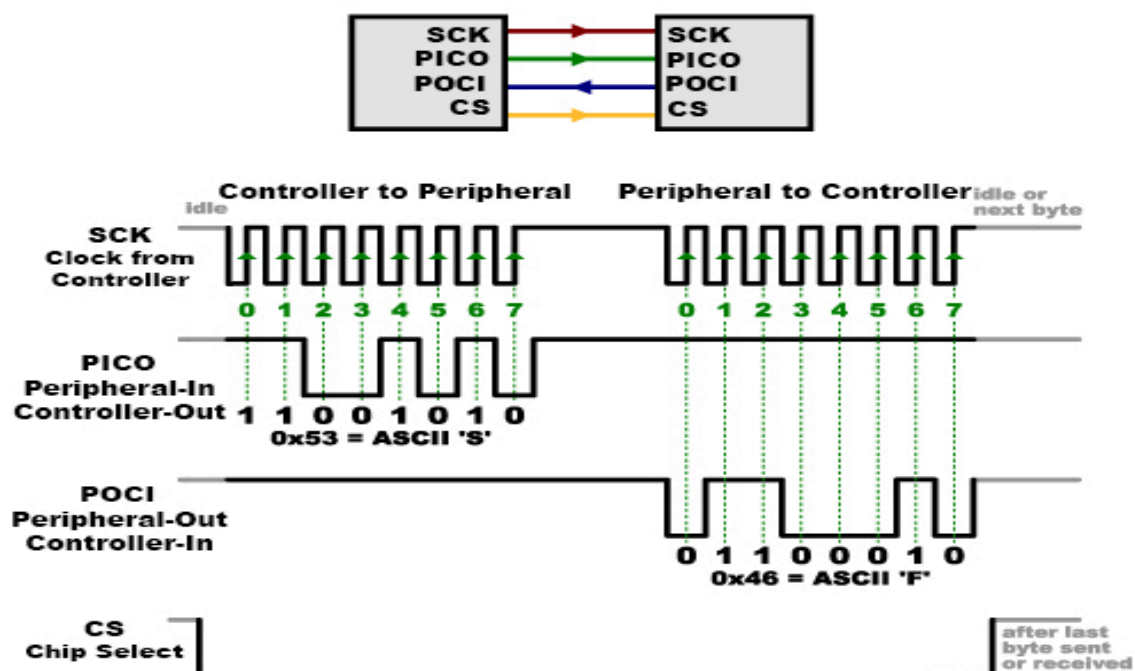
Master can be a microcontroller and the slaves are the i/o devices

Ok one question for everyone - what is the master and the slave in the project we are working on? Answer -

The communication occurs through four main lines:

SPI Bus Lines:

1. SCLK (Serial Clock): Clock signal generated by the master.
2. MOSI (Master Out, Slave In): Data line carrying data from the master to the slave.
3. MISO (Master In, Slave Out): Data line carrying data from the slave to the master.
4. SS / CS (Slave Select / Chip Select): Used by the master to select a specific slave.



💡 Important Note:

- When **CS = LOW**, the slave is active.
- When **CS = HIGH**, the slave is inactive.

SPI operates in four different modes based on two key parameters:

- **Clock Polarity (CPOL)**: Determines whether the clock is idle HIGH (1) or LOW (0).
- **Clock Phase (CPHA)**: Determines whether data is sampled on the rising or falling edge of the clock.

Mode	CPOL	CPHA	Clock Idle State	Data Sampling Edge
0	0	0	LOW	Rising edge
1	0	1	LOW	Falling edge
2	1	0	HIGH	Falling edge
3	1	1	HIGH	Rising edge

Each slave and master must use the same mode for proper communication.

Data Transfer Process in SPI

1. **Master initiates communication** by setting **CS = LOW** for the target slave.
2. **Master generates the clock (SCLK)** to synchronize communication.
3. **Data transmission happens simultaneously in both directions:**
 - Master sends data via **MOSI**.
 - Slave sends data via **MISO** (if required).
4. **Data is shifted bit by bit**, Most Significant Bit (MSB) first.
5. **After transmission, CS is set HIGH**, marking the end of communication.

Advantages of SPI

High-Speed Communication (up to tens of MHz).

Full-Duplex Data Transfer (simultaneous TX & RX).

Simple Hardware Implementation (only 4 wires).

No Start/Stop Bit Overhead (unlike I²C).

SPI vs. Other Communication Protocols

Feature	SPI	I ² C	UART
Number of Wires	4	2	2
Speed	High (~Tens of MHz)	Medium (~400kHz-3.4MHz)	Low (~1Mbps)
Data Transfer	Full-Duplex	Half-Duplex	Asynchronous
Multi-Slave Support	Yes	Yes	No
Complexity	Low	Medium	Low
Best for	Fast, short-range communication	Multiple device communication	Simple serial communication

The basic code for it

```
module spi_master (  
    input wire clk,          // System clock  
    input wire rst,          // Reset signal (active high)  
    input wire [7:0] data_in, // 8-bit data to send  
    input wire start,        // Start signal for transmission  
    output reg sclk,          // SPI Clock  
    output reg mosi,          // Master Out Slave In (MOSI)  
    output reg cs,            // Chip Select (Active Low)  
    output reg done           // Transmission Done Flag  
);  
  
    reg [2:0] bit_count;      // Keeps track of transmitted bits  
    reg [7:0] shift_reg;      // Shift register to send data  
    reg busy;                 // Busy flag to indicate transmission is happening  
  
    always @(posedge clk or posedge rst) begin
```

```

if (rst) begin
    sclk <= 0;      // Reset SPI clock
    cs <= 1;        // Disable chip select (inactive)
    mosi <= 0;      // Clear data output
    done <= 0;      // Clear done flag
    busy <= 0;      // Not busy
    bit_count <= 0; // Reset bit counter
end
else begin
    if (start && !busy) begin
        cs <= 0;      // Activate slave (CS = LOW)
        shift_reg <= data_in; // Load data to shift register
        bit_count <= 0; // Reset bit counter
        busy <= 1;     // Set busy flag
        done <= 0;     // Clear done flag
    end
    else if (busy) begin
        sclk <= ~sclk; // Toggle clock (generates SPI clock)

        if (sclk == 1) begin
            mosi <= shift_reg[7]; // Send MSB first
            shift_reg <= shift_reg << 1; // Shift left
            bit_count <= bit_count + 1; // Increment bit counter

            if (bit_count == 7) begin
                busy <= 0; // Transmission complete
                done <= 1; // Set done flag
                cs <= 1;   // Disable chip select
            end
        end
    end
end
end
end
endmodule

```

Explanation

1. Inputs & Outputs

- **clk**: System clock input.
- **rst**: Active-high reset signal.
- **data_in**: 8-bit data to send to the SPI slave.
- **start**: Start signal to begin transmission.
- **sclk**: SPI clock, controlled by the master.
- **mosi**: Serial data line for sending data.
- **cs**: Chip Select, active low (0 = enabled, 1 = disabled).
- **done**: Goes high when data transmission is complete.

2. Registers Used

- `bit_count`: Counts the number of bits transmitted (0 to 7).
- `shift_reg`: Holds the data and shifts it bit by bit.
- `busy`: Indicates if the SPI is currently transmitting.

3. How the Code Works

- When `rst` is HIGH, all signals are reset.
- When `start` is HIGH and SPI is **not busy**, it:
 - Activates **CS** (selects the slave).
 - Loads `data_in` into `shift_reg`.
 - Starts the transmission.
- The SPI clock (`sclk`) toggles, and with every HIGH edge:
 - The **MSB (Most Significant Bit)** is sent out via `mosi`.
 - The shift register moves the next bit into position.
 - The process continues until all **8 bits** are sent.
- After sending the last bit:
 - `cs` goes HIGH (deactivating the slave).
 - `done` is set HIGH to indicate completion.

How to Use This SPI Master

- Connect `sclk`, `mosi`, and `cs` to an SPI slave.
- Provide `data_in` and set `start = 1` to begin transmission.
- Wait for `done = 1` to know when the data is sent.

Coming to our project

Why Use SPI Over I2C in an Accelerator and FPGA?

When connecting an **accelerator** (like a GPU, TPU, or a dedicated hardware accelerator) to an **FPGA**, SPI is often preferred over I²C for several reasons:

Speed: SPI is Much Faster Than I²C

- **SPI can run at speeds of 50 MHz or higher**, while I²C is typically limited to **3.4 MHz (for High-Speed Mode)**.
- **Accelerators** require **high-speed communication** for real-time processing, which makes SPI a better choice.
- **Example:** If an FPGA is used for real-time **image processing** or **AI acceleration**, fast data transfer is crucial—SPI ensures that data is sent without delays.

Full-Duplex Communication (Faster Data Exchange)

- **SPI is full-duplex** → It **sends and receives data at the same time**.
- **I²C is half-duplex** → It can only send or receive data at one time.

Example: If an accelerator needs to **send results back** to the FPGA while receiving new instructions, SPI allows **simultaneous transmission**, reducing latency.

More Reliable and Stable at High Speeds

- I²C uses **pull-up resistors**, which **limit speed** and **increase power consumption** at high frequencies.
- SPI uses **direct push-pull logic**, making it **more stable** and **less noisy** at higher clock speeds.

Example: If an accelerator is performing **AI inference** or **cryptographic processing**, SPI ensures **stable, high-speed data transfer** without signal distortion.

No Addressing Overhead

- I²C requires **device addressing**, which adds extra **overhead** (extra bits that slow down communication).
- SPI selects devices **directly using Chip Select (CS)**, making transactions **faster and simpler**.

Example: In an FPGA-based **motor control system** with an accelerator, SPI allows **faster commands** to be sent without extra addressing delays.

Easier to Implement in an FPGA

- SPI uses **simple shift registers**, making it easy to implement in an **FPGA using Verilog or VHDL**.

- I²C needs **complex state machines** for arbitration and clock stretching.

Example: If an accelerator is **offloading computation to an FPGA**, implementing SPI in Verilog is simpler than handling I²C's complexity.

Final Answer: Why Use SPI in FPGA + Accelerator?

Higher speed (critical for real-time data processing).

Full-duplex communication (sends & receives data at the same time).

More stable & reliable (no pull-ups, no clock stretching).

Less communication overhead (faster transactions).

Easier to implement in FPGA (simple Verilog/VHDL logic).

What is i2c

I²C (Inter-Integrated Circuit) Protocol – Simple Explanation

What is I²C?

I²C (Inter-Integrated Circuit) is a **two-wire** communication protocol used to connect multiple devices (like sensors, EEPROMs, and microcontrollers) using only **two lines**:

- **SDA (Serial Data Line)** → Transfers data
- **SCL (Serial Clock Line)** → Controls timing

Developed by **Philips (now NXP)**, I²C is widely used in embedded systems, IoT, and FPGA designs.

Features of I²C

Uses only two wires (SDA & SCL)

Can connect multiple devices on the same bus

Supports multiple masters and slaves

Uses addressing (each device has a unique address)

Supports different speeds:

- **Standard Mode** → 100 kHz
- **Fast Mode** → 400 kHz
- **High-Speed Mode** → 3.4 MHz

How I²C Works?

START Condition

- The master **pulls SDA LOW** while SCL is HIGH → This tells all devices that communication is starting.

Addressing Phase

- The master sends a **7-bit or 10-bit address** of the slave.
- The **8th bit (R/W bit)** tells if the master wants to **read (1)** or **write (0)**.

Acknowledge (ACK)

- The slave device checks if the address matches.
- If yes, it pulls **SDA LOW** (ACK) to confirm communication.

Data Transfer

- The master sends or receives **8-bit data**.
- After every byte, the receiver sends an **ACK**.

STOP Condition

- The master **pulls SDA HIGH while SCL is HIGH**, signaling the end of communication.
-

I²C Data Frame Structure

| START | 7-bit Address | R/W | ACK | Data Byte | ACK | STOP |

Example of an I²C Write (Master → Slave)

Master: [START] → [0x50 (7-bit address)] → [0 (Write)] → [ACK]

Master: [Data Byte 1] → [ACK] → [Data Byte 2] → [ACK] → [STOP]

Example of an I²C Read (Slave → Master)

Master: [START] → [0x50 (7-bit address)] → [1 (Read)] → [ACK]

Slave: [Data Byte 1] → [ACK] → [Data Byte 2] → [NACK] → [STOP]

NACK (No Acknowledge) → Used to signal the end of a read operation.

Advantages of I²C Over SPI

Feature	I ² C	SPI
Wires Needed	2 (SDA, SCL)	4+ (MOSI, MISO, SCLK, CS)
Multiple Slaves	Yes, using addresses	Yes, but needs separate CS lines
Speed	Slower (max 3.4 MHz)	Faster (50 MHz or more)
Simplicity	Easier for multiple devices	More complex due to extra lines

Where is I²C Used?

Temperature sensors (e.g., LM75, DS18B20)
OLED displays (SSD1306, SH1106)
RTC (Real-Time Clocks) like DS1307
EEPROM storage (24C02, 24C64)
Communication between FPGA and microcontrollers

How Addressing Works in an I²C Transaction

1. **Master Sends Address**
 - The master sends the **7-bit address + Read/Write bit**.
 - Example: If the **slave's address is 0x50**, the master sends **0xA0** (write) or **0xA1** (read).
2. **Slave Acknowledges (ACK)**
 - If a device with that address exists, it replies with an **ACK** (pulling SDA LOW).
 - If no device responds, the master knows that the address **does not exist**.
3. **Data Transfer Begins**

- If **R/W = 0** → Master writes data to the slave.
- If **R/W = 1** → Slave sends data to the master.

We use i2c in our project for connecting the led

UART (Universal Asynchronous Receiver Transmitter) – Simple Explanation

What is UART?

UART (Universal Asynchronous Receiver Transmitter) is a serial communication protocol used to transfer data **between two devices** using only **two wires**:

- **TX (Transmit)** → Sends data
- **RX (Receive)** → Receives data

It is called **asynchronous** because **no clock signal** is shared between devices. Instead, both devices agree on a common **baud rate** (speed of communication).

Features of UART

Uses only 2 wires (TX & RX)

No clock signal needed (asynchronous communication)

Each device must agree on a baud rate

Full-duplex (can send and receive data simultaneously)

Simple and widely used in microcontrollers, FPGAs, and PCs

How UART Works?

Data Framing (How Data is Sent)

Every data packet in UART consists of the following:

| START Bit | Data (8 bits) | Parity Bit (Optional) | STOP Bit |

- **START Bit (1-bit, LOW/0)** → Signals the beginning of a data transfer
- **Data (5-9 bits, usually 8 bits)** → The actual data being sent

- **Parity Bit (Optional, 1-bit)** → Used for error detection (Even/Odd parity)
- **STOP Bit (1 or 2 bits, HIGH/1)** → Marks the end of data transfer

Baud Rate (Speed of Communication)

Baud rate is the number of bits sent per second. Common baud rates:

- **9600 bps** (slow, commonly used for debugging)
- **115200 bps** (fast, used for real-time data transfer)

Both devices must be set to the **same baud rate** to communicate correctly.

UART Data Transmission Example

Let's say we want to send the letter '**A**' (**ASCII: 65, Binary: 01000001**) at **9600 baud**.

The UART frame looks like this:

| START | 01000001 | PARITY | STOP |

1. **START bit** → LOW (0)
2. **Data** → 01000001 (ASCII 'A')
3. **Parity** → Optional (Even or Odd)
4. **STOP bit** → HIGH (1)

How UART Works Step-by-Step

Idle State: TX and RX are HIGH (1).

Start Condition: TX line goes LOW (0) → Signals the start of transmission.

Data Transfer: Bits are sent one by one at the agreed baud rate.

Stop Condition: TX line goes HIGH (1) → Marks the end of transmission.

Feature	UART	I ² C	SPI
Wires Needed	2 (TX, RX)	2 (SDA, SCL)	4+ (MOSI, MISO, SCLK, CS)
Clock Signal	No (Asynchronous)	Yes (SCL)	Yes (SCLK)
Speed	Medium (~115200 bps)	Slow (3.4 MHz max)	Fast (50 MHz+)

Multiple Devices	No	Yes (Addressing)	Yes (Chip Select)
Complexity	Simple	Medium	Complex

UART vs. Other Communication Protocols

Where is UART Used?

Connecting microcontrollers (Arduino, ESP32, Raspberry Pi) to a PC
 Debugging using Serial Monitors (like PuTTY, Arduino Serial Monitor)
 Wireless modules (Bluetooth, GPS, LoRa, GSM)
 FPGA communication (sending debug data to a PC terminal)

Summary

UART uses only 2 wires (TX & RX) for communication
 No clock signal needed (asynchronous communication)
 Each device must agree on a baud rate
 Simple, widely used in microcontrollers, FPGAs, and PCs
 Used for debugging, wireless communication, and serial interfaces